DeepL
订阅DeepL Pro以翻译大型文件。
欲了解更多信息，请访问www.DeepL.com/pro。

系统软件中BST的性能分析

[扩展摘要]

Ben Pfaff
斯坦福大学计算机科学系
blp@cs.stanford.edu

类别和主题描述符：E.1 [数据结构]: 树; E.2 [数据存储表示法]: 链接表示法; C.4 [系统的性能]: 性能属性

通用术语：算法、性能

关键词：二进制搜索树，BST，线程树，AVL树，红黑树，Splay树

1. 简介

操作系统内核经常使用基于二进制搜索树（BST）的数据结构。选择正确的树和节点表示法可以大大影响使用这些数据结构的代码的性能。令人惊讶的是，对于用于管理基于BST的数据结构的算法与实际系统中的性能特征之间的关系，几乎没有任何仿真研究[6, 2, 10]。这篇扩展论文概述了20个BST变体的性能结果：四个数据结构变体各有五个不同的节点表示。

2. 数据结构

在普通的BST中，以病态的顺序插入数据项，如排序的顺序，会使n节点树的每次操作性能从$O(\lg n)$下降到$O(n)$[7]。一种解决方案是平衡树，如*AVL树*或*红黑树*，它们将n *节点*树的高度分别限制在不超过$1.4405 \lg(n + 2) - 0.3277$和$2 \lg(n + 1)$ [1, 5]。另一个流行的解决方案是*splay树*，它在访问时将每个节点旋转或 "splay "到根上[9]。

一个最小的BST节点包含一个数据项和左右两个子指针，但是在这样一个*普通的BST*中，有效的按顺序遍历需要维护一个堆栈，如果在遍历过程中树被修改，通常必须重建堆栈。消除堆栈的一个方法是给节点添加一个父指针，产生一个*带有父指针的树*。为每个节点添加前任和后任指针，产生一个*链接列表树*，是另一种解决方案。另外，我们可以使用本来是空的右子指针来存储节点的继任者的指针，产生一个*右线程树*。如果我们也使用原本为空的左子指针来存储前任节点，结果就是一棵*线程树*。

版权由作者/所有者持有。

SIGMETRICS/Performance'04, June 12-16, 2004, New York, NY, USA.美国医学会1-58113-664-1/04/0006。

3. 实验

我们做了三个实验来测试上一节中描述的二进制搜索树的20种变化。

3.1 VMAs

我们的第一个实验测量了管理一组虚拟内存区（VMAs）的速度，这些虚拟内存区代表了Unix进程中的内存部分。一个正常的进程至少有三个VMA，用于其代码、数据和堆栈段，但进程可以通过mmap系统调用创建一个任意数量的VMA。由于VMA的大小从4 kB到超过1 GB不等，哈希表不是一个合适的表示。大多数类似Unix的操作系统使用BST的变体来跟踪VMA，Windows NT也对其VMA等价物进行跟踪[4]。

我们记录了三个真实程序中的VMA活动：Mozilla 1.0，VMware GSX Server 2.0.1，以及在用户模式Linux 2.4.18.48下运行的squid。这三个测试集显示了非常不同的VMA行为，但它们都显示了很强的定位性。我们在VMA活动的模拟中使用这些测试集。我们用20个BST变体中的每一个对每个测试集进行了模拟。

Splay树是所有三个数据集的大赢家，每次都比最好的竞争者快23%到40%，这是因为Splay树有能力优化树的参考位置性。AVL树的速度一直比红黑树快，最多可达20%。AVL树的比较次数也少了32%，这表明是由于更严格的AVL平衡规则造成的。然而，红黑树的平均内部路径长度只比AVL树长3%左右。我们的结论是，AVL和红黑树的全局平衡性差不多，但是AVL平衡规则对于像这些具有强地方性的测试集来说，在重要的地方导致了更好的平衡性。除两种情况外，普通BST是所有情况下最慢的节点表示。在平衡树的实现中，普通节点是最慢的，因为在修改树后重建堆栈而拖累了它，而父指针是最快的，因为它们在区分子指针和前任指针方面的额外开销而击败了线程表示法。

或继承人。

3.2 互联网同行缓存

RFC 791[8]要求网间主机发送的每个IP包都要有一个16位的识别字段，"对于该源-目的对来说必须是唯一的，并且在数据报在互联网系统中活动的时间内是原生的"。Linux内核通过维护每个主机计数器的AVL树来确保这一点，增加一个主机的计数器

每传输一个数据包，就会有一次。散列表在一般情况下会更快，但攻击者可以造成散列碰撞，从而降低其性能。Univer- sal散列[3]可以使这种攻击更加困难，但它们在平衡树中是不可能的。

对等的AVL树被用作第二级缓存。第一级缓存是路由缓存，其中每个条目都包含一个指向相应的对等AVL树节点的指针。当路由缓存中的一个条目被丢弃时，它在对等AVL树中的对应节点在被丢弃之前会保留一定的时间。另一方面，如果在这段时间内为该IP地址添加了一个新的路由缓存条目，相应的对等AVL树节点将被保留。

我们写了一个模拟，试图从法律上重现路由缓存的最重要特征。我们在两种情况下对其行为进行了实验，尝试了所有的
每个场景中有20个BST变体。第一个场景是 "正常 "负载下的网络服务器，其中随机IP地址的主机平均在30秒的间隔内发出9个请求。对于这个数据集，通常父指针和线程节点是最快的，反映了这些类型的节点允许$O(1)$删除。在节点代表中，速度是一致的，从最快到最慢，或分为不平衡树、红黑树、AVL树和splay树，反映了每一种树在试图平衡已经随机的数据时所进行的额外工作。

第二种情况是一种 "攻击"，在这种情况下，连续编号的IP地址每10毫秒就会发出一个新的请求。对于这种情况，AVL树比红黑树表现得更好，因为更严格的AVL平衡规则能更好地保持树的高度在重要的一点，即下一个插入点的最小值。

3.3 交叉参考资料整理器

前两个实验处理的问题最好用BST来解决。然而，在实践中，那些用哈希表更好地解决的问题有时也会用BST来解决。例如，C++98标准库提供了（典型的）基于BST的 "set "和 "map "模板类，但没有基于哈希的模板类，鼓励C++程序员使用错误的工具。

考虑到这一点，我们编写了一个 "交叉引用整理器 "工具，该工具使用BST作为索引对从程序源代码中提取的一组标识符进行排序和合并。我们在Linux 2.4.18的include/linux文件集上用所有20种BST变体运行了这个工具。为了测试最坏和最好的情况下的顺序，我们还在同一测试集的分类和随机洗牌的标识符集合上运行了它。

对于正常情况，splay树是一贯的赢家，利用了局部性的优势，这是因为标识符在首次提及的几行内有重复出现的趋势。红黑树始终比AVL树稍快，反映了更宽松的红黑平衡规则，导致更少的再平衡。最后，不平衡树的时间最多只比AVL树多11%，表明自然标识符的排序倾向于避免随机性。

Splay树也是排序情况下的大赢家，这也是因为它们在位置性方面的优势。AVL树比红黑树表现更好，因为更严格的平衡规则保持了更短的最大路径长度。

在洗牌的情况下，做了较少额外工作的树再次成为赢家。然而，洗牌后的结果是一致的。比相应的 "正常 "结果慢得多。我们发现，洗牌后的结果缺乏位置性，加上处理器缓存的大小有限，是罪魁祸首。

4. 结论

该扩展摘要总结了我们对20种BST变体在三种不同的实际情况下的性能进行的经验比较，并证明了数据结构的选择会对性能产生重大影响。我们的结果表明，BST数据结构和节点代表应根据输入的预期模式和要执行的操作组合来选择。

我们发现，在选择数据结构时，当可以依赖随机排序的输入时，不平衡BST是最好的；如果随机排序是常态，但预计偶尔会有排序的运行，那么应该选择红黑树。另一方面，如果插入经常以排序的顺序发生，那么当后面的访问倾向于随机时，AVL树表现出色，而当后面的访问是顺序的或集群的，splay树表现最好。

对于节点表示法，我们发现父指针通常是最快的，所以只要每个节点的额外指针字段的成本不重要，它们就应该是首选。如果空间有限，线程表示法可以提供内存，在速度上仅略微落后于父指针。一个普通的BST有较少的字段需要更新，但结合遍历和修改需要额外的搜索。当遍历非常普遍时，维护一个按顺序排列的节点的链表是一个很好的选择，但需要很高的内存成本。最后，在我们所有的实验中，右线程表示法的表现很差。

5. 鸣谢

斯坦福大学研究生奖学金资助了这项工作。

6. 全文

请访问`http://benpfaff.org/papers`，查看本扩展摘要所总结的论文全文。

7. 参考文献

[1] G. M. Adel'son-Vel'skiˇı和E. M. Landis。信息组织的一种算法。*Soviet Mathematics Doklady*, 3:1259-1262, 1962.

[2] J.-L. Baer and B. Schwab.树形平衡算法的比较.*Communications of the ACM*, 20(5):322-330, 1977.

[3] T.H. Cormen, C. E. Leiserson, and R. L. Rivest.*算法入门*，第12.3.3节，第229-232页。麻省理工学院出版社，1990年。

[4] H.卡斯特，编辑。*Inside Windows NT*，第200页。微软出版社，1993年。

[5] L.J. Guibas and R. Sedgewick.平衡树的二色性框架。在*第19届计算机科学基础年度研讨会论文集中*，第8-21页，1978。

[6] P.L. Karlton, S. H. Fuller, R. E. Scroggs, and E. B. Kaehler.高度平衡的树的性能。*Communications of the ACM*, 19(1):23-28, 1976.

[7] D.E. Knuth.*排序和搜索*，*《计算机编程艺术》*第三卷，第6.2.2节，第430-31页。Addison-Wesley, Reading, Massachusetts, 第二版，1997。

[8] J.Postel.RFC 791：互联网协议，1981年9月。状态：STANDARD。

[9] D.D. Sleator and R. E. Tarjan.自我调整的二进制搜索树。*ACM杂志*，32（3）：652-686，1985年7月。

[10] W.E. Wright.动态维护二进制搜索树的算法的经验评估。在*ACM 1980年年会的论文集中*，第505-515页，1980年。